Low Level Design Document

Introduction

This Low Level Design (LLD) document details the implementation plan for **AgentX - Multi-Agent Collaboration Framework**. AgentX is a Python-based system leveraging LangGraph, Langchain, and FastAPI to enable specialized AI agents to collaboratively solve complex legal tasks via dynamic, context-aware workflows.

---

1. System Components

Component	Technology	Key Responsibilities
API Layer	FastAPI	Expose endpoints, receive tasks, return results
Orchestrator	LangGraph	Manage agent workflow, task routing, state management
Agent Manager	Langchain	Instantiate/configure agents, manage agent registry
Specialized Agent	Langchain/OpenAI	Execute domain-specific subtasks, maintain memory
Memory Store	In-memory/DB	Persist agent context, conversation history

---

2. Class/Interface Overview

Class/Interface	Description	Key Methods/Attributes
`AgentOrchestrator`	Manages agent workflow and task assignment	`run_workflow()`, `route_task()`, `state`
`AgentManager`	Registers and configures agents	`register_agent()`, `get_agent()`
`LegalAgent`	Base class for specialized agents	`process_input()`, `memory`
`MemoryStore`	Stores/retrieves agent context	`save_context()`, `load_context()`
`APIServer`	FastAPI app, exposes endpoints	`POST /task`, `GET /status`

Relationships:

- `APIServer` → `AgentOrchestrator` → `AgentManager` → `LegalAgent`
- `LegalAgent` ↔ `MemoryStore`

---

3. Data Structure Overview

Model	Fields/Attributes	Purpose
`TaskRequest`	`task_id`, `input_data`, `metadata`	Incoming task payload

`AgentState`	`agent_id`, `context`, `history`, `status`	Tracks agent execution state
`AgentResponse`	`task_id`, `output_data`, `status`, `errors`	Agent output/result
`MemoryRecord`	`agent_id`, `conversation_id`, `data`, `timestamp`	Persisted memory/context

---

4. Algorithms/Logic

Agent Orchestration Flow (Pseudocode):

```
def run_workflow(task_request):
    state = initialize_state(task_request)
    while not state.is_complete():
        agent = select_next_agent(state)
        response = agent.process_input(state.context)
        state.update(response)
        MemoryStore.save_context(agent.id, state.context)
    return state.final_output()
```

---

5. Error Handling

Scenario	Handling Strategy
Invalid API request	Return HTTP 400 with error message
Agent processing failure	Log error, update status, return fallback
Memory store unavailable	Retry, else return error to orchestrator
External API (OpenAI) failure	Exponential backoff, fallback response
Workflow timeout	Abort workflow, return partial results

End of Document